

SUPERBOARD

Comprehensive Feature QA Audit Report

Full-Stack Application Testing: Design, Functionality, Security, and UX

Application	Superboard (Tutoring SaaS Platform)
Stack	Next.js 16, Supabase, Prisma, LiveKit, Stripe
Date	August 10, 2026
Scope	60+ API Routes, 20+ Dashboard Panels, Full Landing Page, Canvas/Whiteboard
Findings	4 Critical, 15 High, 26 Medium, 22 Low

Table of Contents

1. Executive Summary	4
Finding Severity Summary	4
2. Landing Page and Authentication	5
2.1 Landing Page (LandingPage.tsx - 862 lines)	5
2.2 Authentication Flow	5
3. Dashboard and Navigation	6
3.1 Dashboard Shell (DashboardPage.tsx - 1087 lines)	6
3.2 My Rooms Panel (MyRoomsPanel.tsx)	6
4. Canvas and Whiteboard	7
4.1 Whiteboard Core (Whiteboard.tsx)	7
4.2 Toolbar (Toolbar.tsx - 442 lines)	7
4.3 Session Timer (SessionTimer.tsx)	7
4.4 Page Sidebar (PageSidebar.tsx)	7
5. Billing and Subscriptions	9
5.1 Billing Panel (BillingPanel.tsx)	9
5.2 Stripe Integration (API)	9
6. Agency Management	10
6.1 Agency Admin Panel (AgencyAdminPanel.tsx)	10
6.2 Invite Page (invite/[code]/page.tsx)	10
7. Recordings Panel	11
7.1 RecordingsPanel.tsx (445 lines)	11
7.2 Recording API - Critical Findings	11
8. Security Deep Dive	12
8.1 Middleware Security (middleware.ts)	12
8.2 Rate Limiting (rate-limit.ts)	12
8.3 Admin API Validation	12
9. Scheduling, Homework, and Invoices	14
9.1 Schedule Panel (SchedulePanel.tsx - 533 lines)	14
9.2 Homework Panel (HomeworkPanel.tsx - 746 lines)	14
9.3 Invoice Panel (InvoicePanel.tsx - 619 lines)	14
10. Parent Portal and Student Experience	15

10.1 Parent Portal (parent/[token]/page.tsx - 488 lines)	15
10.2 Student Dashboard (StudentDashboard.tsx - 614 lines)	15
11. Analytics and Data Visualization	16
11.1 Analytics Panel (AnalyticsPanel.tsx)	16
11.2 Agency Analytics (AgencyAnalyticsPanel.tsx)	16
12. Cross-Cutting Patterns and Systemic Issues	17
12.1 Design System Consistency	17
12.2 Accessibility Summary	17
12.3 N+1 API Call Pattern	17
12.4 Silent Error Swallowing	18
13. Priority Remediation Roadmap	19
Phase 1: Critical Fixes (Immediate - 1-2 days)	19
Phase 2: High Priority (1 week)	19
Phase 3: Medium Priority (2-3 weeks)	19
Phase 4: Polish and Low Priority (Ongoing)	20
14. Conclusion	21

1. Executive Summary

Superboard is a full-featured online tutoring platform built with Next.js 16, Supabase PostgreSQL, Prisma ORM, LiveKit for real-time video/audio, Hocuspocus for CRDT-based whiteboard collaboration, and Stripe for subscription billing. The platform serves three primary personas: individual tutors, tutoring agencies (with sub-tutor management), and students/parents. This report presents findings from a comprehensive feature-by-feature audit covering UI/UX design, functional correctness, security posture, API robustness, accessibility, and performance patterns.

The audit examined 60+ API route files, 20+ dashboard panel components, the full marketing landing page, authentication flows, middleware security, canvas/whiteboard integration, billing and subscription management, agency administration, scheduling, homework and invoicing systems, the parent portal, and the invite system. Each feature was evaluated on a Good/Bad/Ugly/Remedy framework to provide actionable, prioritized remediation guidance.

Overall, Superboard demonstrates above-average engineering quality for a startup SaaS product. The design system is cohesive, the security architecture includes multiple deliberate hardening layers (CSP nonces, double-submit CSRF, rate limiting, HMAC-signed URLs), and the feature set is comprehensive. However, the audit identified 4 Critical, 15 High, 26 Medium, and 22 Low severity findings that require attention, spanning broken dark mode support, missing destructive action confirmations, race conditions in billing, silent error swallowing, and several security gaps in authentication and rate limiting.

Finding Severity Summary

Severity	Count	Primary Themes
CRITICAL	4	CSRF bypass, Recording secret instability, JWT in localStorage, LiveKit webhook falthrough
HIGH	15	Missing admin validation, Broken agency access checks, Rate limit bypass, No destructive confirmations, N+1 API calls
MEDIUM	26	Dark mode broken, Inconsistent loading patterns, Race conditions, Missing ARIA attributes, Overly permissive access checks
LOW	22	Code quality (any types, silent catches), DRY violations, Non-standard Tailwind classes, Missing pagination

2. Landing Page and Authentication

2.1 Landing Page (LandingPage.tsx - 862 lines)

GOOD: Strong hero section with emerald gradient text, trust signals, and a dual-CTA layout (Sign Up / Log In). The mini whiteboard preview in the hero gives immediate visual context about the product. The navigation is fixed with mobile hamburger menu. The feature grid, pricing comparison table (4 plans with monthly/annual toggle), FAQ accordion, and social proof sections create a complete marketing funnel. Google OAuth integration uses a custom SVG icon maintaining brand consistency.

BAD: The 862-line monolithic file bundles the entire landing page AND all authentication logic (login, register, Google OAuth, forgot password, password strength meter) into a single component. Auth logic should be extracted into a dedicated AuthDialog component. The navbar has no backdrop blur, so scrolled content shows through a flat white background. The mobile hamburger menu does not close on outside-click despite a mobileMenuRef being declared. Auth submit buttons have no loading spinners during authentication requests, leading to potential double-submission. The "Contact Sales" button on the Agency Premium plan uses low-emphasis outline variant despite being the only CTA for that tier.

UGLY: Testimonials section uses placeholder names ("Sarah C.", "Michael T.") with no real photos or verifiable identities. For an MVP this is acceptable but should be disclosed or replaced before launch. The password strength indicator uses 5 segments but provides no "minimum 6 characters" helper text, so users see "Weak" without understanding why.

REMEDY: Extract AuthDialog into src/components/auth/AuthDialog.tsx. Add backdrop-blur-md and bg-white/80 to the navbar. Implement outside-click detection using mobileMenuRef for mobile menu. Add loading spinners to all auth submit buttons. Change Agency Premium CTA to a more prominent style. Add "min 6 characters" helper text to password fields. Consider adding a password visibility toggle.

2.2 Authentication Flow

GOOD: Google OAuth via Supabase Auth is properly integrated with custom SVG branding. The auth gate in page.tsx correctly routes users based on authentication state (landing, dashboard, or admin). Supabase client-side handles JWT caching and automatic refresh. The onAuthStateChange listener ensures real-time auth state synchronization across browser tabs.

BAD: Multiple silent catch blocks in the auth gate (page.tsx lines 68, 80, 81, 109, 110) completely swallow errors. Auth failures, profile fetch failures, and admin check failures are all silently ignored with no console logging. The same auth check logic is duplicated between the onAuthStateChange callback and the initial getUser().then() chain, causing redundant API calls on every mount. JWT tokens are stored in localStorage (Supabase default), making them accessible to any XSS attack. The CSRF double-submit pattern in middleware has a conditional validation bug: the check is skipped entirely when either the cookie or header is missing, rather than rejecting the request.

REMEDY: Add console.warn() at minimum to all catch blocks. Extract duplicated auth logic into a shared function (e.g., fetchUserProfile()). Plan migration from localStorage JWT to httpOnly cookie-based auth (noted as a known gap in auth-fetch.ts comments). Fix CSRF middleware to reject when either cookie or header is missing on state-changing requests (line 81).

3. Dashboard and Navigation

3.1 Dashboard Shell (DashboardPage.tsx - 1087 lines)

GOOD: Clean sidebar information architecture with grouped navigation (Main, Workspace, Resources, Team, Account). Collapsible sidebar with tooltip fallbacks on collapsed state. Mobile sheet-style menu slides from left. Active route indicator with green dot. Consistent rounded-2xl card containers. Welcome banner with gradient hero and quick stats. Usage stat cards with distinct gradient backgrounds per metric type. Quick Start subject grid with proper icons per subject area. Sign-out uses AlertDialog for destructive action confirmation (one of the few places that does).

BAD: Dark mode is fundamentally broken: the sidebar hardcodes bg-white and border-gray-200 with no dark: variants. The top bar uses bg-white/80. Cards inside SettingsPanel use bg-gray-50 and text-gray-600. The dark mode toggle in Settings actually works (toggles .dark class), but 80%+ of components use literal gray colors instead of semantic CSS tokens (bg-card, text-foreground, border-border), so dark mode looks completely broken. The student-progress view renders a blank main area when activeView is student-progress but no student is selected. The Resources view has duplicated Tabs JSX between agency and non-agency code paths sharing ~20 identical lines of markup.

UGLY: Sidebar collapse button is a raw HTML button element rather than the shadcn Button component, causing icon alignment inconsistencies. The sidebar uses fixed positioning on mobile and sticky on desktop, creating two CSS positioning contexts that can fight during viewport resize transitions.

REMEDY: Replace all hardcoded bg-white, text-gray-*, border-gray-* with semantic tokens (bg-card, text-foreground, border-border). Either fully implement dark mode across all components or remove the toggle to avoid false advertising. Extract Resources Tabs into a shared component. Change sidebar collapse button to Button variant="ghost". Guard student-progress view with redirect when no student is selected.

3.2 My Rooms Panel (MyRoomsPanel.tsx)

GOOD: Clean loading skeleton with 3 animated placeholder rows. Proper empty state with GraduationCap icon and CTA button. Relative date formatting ("Today", "Yesterday") for recent rooms. Subject-specific icons with gradient backgrounds. Active/Ended status badges. Action buttons hidden on desktop hover but always visible on mobile (opacity-100 sm:opacity-0). End lesson button shows Loader2 spinner during the API call.

BAD: No search or filter capability on the room list. With many rooms, the max-h-96 overflow-y-auto container becomes unusable. Ending a room has NO confirmation dialog despite being a destructive action. One click on the X icon immediately calls handleEndRoom. Each room shows generic text like "Math Lesson" with no user-editable title, making it impossible to distinguish between multiple rooms of the same subject.

REMEDY: Add AlertDialog confirmation before ending a room. Add room titles (editable) and display them in the list. Add basic filtering (active/ended toggle or search input).

4. Canvas and Whiteboard

4.1 Whiteboard Core (Whiteboard.tsx)

GOOD: Proper lazy loading of 12+ heavy components via Next.js dynamic() imports, keeping initial bundle size manageable. CRDT-based collaboration via Yjs and Hocuspocus ensures conflict-free real-time sync. Student waiting room flow requires name entry before participation. Tool mapping layer translates app-specific tool IDs to Tldraw internal IDs, creating a clean abstraction boundary.

BAD: No loading state while Tldraw canvas loads. The canvas area is completely blank from the time the component mounts until the lazy import hydrates, which can take 1-3 seconds on slow connections. No error boundary wrapping the TldrawCanvas component. If the canvas crashes (known to happen with complex drawings or memory pressure), the entire room page breaks with no recovery option.

REMEDY: Add a branded loading overlay (matching the app spinner pattern) that covers the canvas area during hydration. Wrap TldrawCanvas in a React ErrorBoundary with a "Restart Canvas" recovery button.

4.2 Toolbar (Toolbar.tsx - 442 lines)

GOOD: Responsive design: desktop vertical sidebar toolbar, mobile floating bottom bar plus a "More" sheet for overflow tools. Proper aria-label and role="toolbar" ARIA attributes. Subject-specific toolkits are lazy-loaded based on room subject. AI tools are gated behind tier checks. Pro-only tools are labeled in the SUBJECT_AI_TOOLS configuration.

BAD: AI tool buttons do not pre-select the specific AI action when clicked. Clicking any AI tool just opens the AI panel without passing the selected tool action, so users have to manually select the action again inside the panel. No visual distinction between free and pro AI tools in the toolbar; they look identical until clicked. Desktop toolbar buttons are very narrow (w-9) with no visible labels, relying entirely on tooltips.

UGLY: Uses w-4.5 h-4.5 which is not a standard Tailwind class. It works in Tailwind v4 but is non-standard and may confuse developers.

REMEDY: Pass the selected action to the AI panel when a specific AI tool button is clicked. Add a small lock icon overlay on pro-only tools. Consider adding a text-label toggle option for accessibility on the desktop toolbar.

4.3 Session Timer (SessionTimer.tsx)

GOOD: 5-minute warning with amber pulse animation provides clear visual urgency. Monospace tabular-nums font ensures stable number width preventing layout shifts. Extend +15min and End buttons are tutor-only. Backdrop blur on the floating timer container maintains readability over any canvas content.

BAD: The "End" button has no confirmation dialog. One click immediately ends the lesson, potentially losing unsynchronized whiteboard data and abruptly disconnecting students. The extend button uses a spinning Plus icon during extension, which visually mimics a loading spinner rather than communicating "adding time".

REMEDY: Add AlertDialog confirmation for the End button. Change the extend icon to Clock with "+15" text for clearer affordance.

4.4 Page Sidebar (PageSidebar.tsx)

GOOD: Tutor-only visibility (students see nothing, maintaining teaching control). ScrollArea component handles many pages gracefully. Add/delete buttons with min-page guard (always keeps at least 1 page).

BAD: Pages are displayed as plain numbered buttons (w-12 h-16) with no thumbnail or preview. Users cannot visually identify which page contains what content. The delete button is always visible when pages > 1, even for the currently active page, risking accidental deletion of the page being worked on. No drag-to-reorder capability.

REMEDY: *Disable the delete button on the current active page. Add drag-to-reorder using a drag-and-drop library. Consider generating simplified page thumbnails.*

5. Billing and Subscriptions

5.1 Billing Panel (BillingPanel.tsx)

GOOD: Clean tier-based upgrade flow showing Free, Pro, Agency Standard, and Agency Premium tiers. "Popular" and "Best Value" badges on upgrade cards help guide decisions. Feature comparison lists with check icons. White-label brand color input includes a live swatch preview. Disabled custom domain field with "Contact Sales" link for enterprise inquiries. Legal notice section links to Terms, Privacy, and Refund Policy.

BAD: No loading state on upgrade buttons. `handleUpgrade` opens a Stripe checkout via `window.open`, but if the network is slow, the user may click multiple times creating multiple checkout sessions. The brand color input has no hex validation. Users can type "not-a-color" and the swatch shows `backgroundColor` as transparent/white with no error feedback. The brand color swatch preview always shows gradient-primary (emerald-to-teal) regardless of the actual `brandColor` value. No current billing period or next payment date is displayed anywhere.

REMEDY: Add hex color validation using regex `^[0-9a-fA-F]{6}$` with inline error. Add loading/disabled state to upgrade buttons during Stripe session creation. Show the swatch with the actual `brandColor` value. Display current billing period dates on the plan card.

5.2 Stripe Integration (API)

GOOD: Checkout sessions are created server-side with proper metadata (`userId`, `tier`). Webhook endpoint verifies Stripe signatures using the signing secret. The billing panel correctly reads the current plan from the profile and shows appropriate upgrade options.

BAD: Stripe Checkout GET endpoint (`stripe/checkout/route.ts:74`) leaks internal error messages by returning `error.message` directly to the client. This could expose Stripe API configuration details, internal stack traces, or database state. The Admin Users GET endpoint exposes `stripeCustomerId` in the API response, which is sensitive payment processor data even for admin eyes.

REMEDY: Log the real Stripe error server-side and return a generic "Unable to create checkout session" message. Remove `stripeCustomerId` from the default admin users select; add a separate admin detail endpoint if this data is truly needed.

6. Agency Management

6.1 Agency Admin Panel (AgencyAdminPanel.tsx)

GOOD: Sub-tutor count with max limit display (e.g., "3/5" or "3/infinity"). Upsell warning banner appears at the limit. Invite flow has clear UX: email input, success state with copyable invite link, and "Send Another" action. Invite list shows status badges (Pending, Expired, Cancelled, Accepted) with distinct colors. Sub-tutor table displays per-user statistics (rooms, lessons). Inline remove confirmation uses "Sure?" prompt with Yes/No buttons.

BAD: Loading state is just a bare spinner with "Loading..." text, lacking the skeleton pattern used by other panels. No pagination on the sub-tutors table. Stats grid uses grid-cols-4 with no responsive breakpoint, causing 4 columns to be extremely cramped on mobile. The "Sure?" inline confirmation for remove is not a proper AlertDialog and lacks the visual weight needed for destructive actions. The copyInviteLink function uses a Download icon for the copy action, which is semantically incorrect.

REMEDY: Use AlertDialog for sub-tutor removal confirmation. Add grid-cols-2 md:grid-cols-4 responsive breakpoint. Use Copy icon instead of Download for copy actions. Add loading skeletons. Add pagination to sub-tutors table.

6.2 Invite Page (invite/[code]/page.tsx)

GOOD: Excellent state machine with 5 distinct states: loading, not_found, expired, already_used, and loaded. Each state has appropriate visuals, icons, colors, and CTAs. Agency branding is dynamically applied using the agency custom color as a gradient header. The accept flow checks for email match between authenticated user and invite, showing a clear mismatch warning if they differ. Accept error is shown inline below the button.

BAD: Loading state is a bare spinning div with no text label, making it invisible to screen readers. Agency logo uses img tag without an onError handler for broken images. Accept button loading state uses a hand-rolled spinner instead of the standard Loader2 icon, and lacks aria-busy="true".

REMEDY: Add aria-label="Loading invite..." to the loading spinner. Add onError handler to agency logo img to show a fallback icon. Use Loader2 icon for consistent loading patterns. Add aria-busy to accept button during submission.

7. Recordings Panel

7.1 RecordingsPanel.tsx (445 lines)

GOOD: Best-in-class state handling among all dashboard panels. Has four distinct states: paywall (for free users), loading skeleton, error with retry button, and empty state with CTA. Status badges for Recording, Completed, and Failed with distinct semantic colors. Inline video player overlay with backdrop blur and close button. Download action for completed recordings. Disabled states on Processing/Failed recordings with descriptive text explaining why.

BAD: N+1 API call pattern: fetches all rooms, then for each ended room, fires individual recording fetch requests via Promise.all. With 50 ended rooms, this triggers 51 API calls on mount, creating significant load time. The video overlay is a raw fixed div rather than the shadcn Dialog component, missing keyboard focus trapping and ESC-to-close behavior. Tab navigation can escape behind the overlay.

REMEDY: Create a batch recordings endpoint (e.g., GET /api/recordings?tutorId=...) to eliminate N+1. Use the Dialog component for the video player to inherit focus trapping, ESC handling, and backdrop click-to-close.

7.2 Recording API - Critical Findings

[CRITICAL]

BAD: Recording URL sign secret (RECORDING_URL_SIGN_SECRET) defaults to crypto.randomBytes(32).toString("hex") when the environment variable is missing. In a serverless environment (Vercel), each cold start generates a new random secret, immediately invalidating all previously issued signed recording download URLs. Users who bookmarked or received download links will find them permanently broken after any deployment. LiveKit webhook handler at livekit/webhook/route.ts:107 has the success return statement nested inside the egress_failed block. After successfully processing an egress_ended event, execution falls through past the egress_failed block to the outer catch, returning a 500 error. This causes LiveKit to retry egress_ended events, potentially re-processing recordings or corrupting state.

REMEDY: Fail loudly at startup if RECORDING_URL_SIGN_SECRET env var is not set (throw an error during module initialization). Move the return NextResponse.json({ received: true }) statement outside both if blocks so it executes after either egress_ended or egress_failed handling succeeds.

8. Security Deep Dive

8.1 Middleware Security (middleware.ts)

GOOD: Multi-layered security architecture with CSP nonce generation per request using Web Crypto API. Double-submit CSRF pattern implemented with crypto-random token in cookie and matching X-CSRF-Token header. Per-path rate limiting via Upstash Redis with in-memory fallback. Custom domain routing for agency branding. API routes receive strict CSP (default-src none, frame-ancestors none). Webhook and Stripe paths correctly bypass CSRF in favor of HMAC signature verification.

BAD: CSRF check at line 81 is conditional: `if (!isWebhook && csrfCookie && csrfHeader)`. When either the cookie or header is missing, the check is silently skipped instead of rejecting the request. An attacker can send POST requests from a different origin where the CSRF cookie was never set, bypassing protection entirely. CSRF comparison uses `!==` (non-timing-safe) despite the comment claiming "Constant-time comparison". The `crypto.timingSafeEqual` pattern already exists correctly in `livekit/webhook/route.ts` but was not applied here. No server-side route protection: authenticated pages (`/dashboard`, `/room/*`) are served to anyone; auth gating is handled entirely client-side. The HTML shell, JS bundles, and any server-rendered content are delivered to unauthenticated users.

REMEDY: Fix CSRF validation to reject when either cookie or header is missing on state-changing requests. Replace `!==` with `crypto.timingSafeEqual()` for CSRF comparison (pattern already exists in the codebase). Consider adding middleware-level auth checks for sensitive routes to prevent serving authenticated content to unauthenticated users.

8.2 Rate Limiting (rate-limit.ts)

GOOD: Tiered architecture with Upstash Redis as primary and in-memory Map as fallback. Category-specific limits (livekit 10/min, auth 20/min, ai 30/min, participants 50/min, parentPortal 5/15min). Proper IP extraction chain: X-Real-IP, X-Forwarded-For (with trusted proxy check), fallback to "unknown". 5-minute periodic cleanup prevents unbounded memory growth in fallback mode.

BAD: When `extractClientIP` returns "unknown" (no proxy headers present), `checkRateLimit` returns `{ allowed: true, remaining: 999 }`, effectively bypassing all rate limiting. An attacker behind a misconfigured proxy or using direct connections can make unlimited requests. X-Forwarded-For is accepted without mandatory trusted proxy configuration, allowing IP spoofing by setting the header manually.

REMEDY: Apply a conservative default limit (e.g., 10/min) when IP is "unknown" instead of a free pass. Make trusted proxy configuration mandatory in production. Add rate limiting to currently unprotected endpoints: AI actions, fingerprint checks, recording starts, webhook registration, and student import.

8.3 Admin API Validation

GOOD: Admin endpoints use `requireAdmin()` middleware for authentication. The PATCH endpoint for users validates tier against `VALID_TIERS`. Audit logging captures admin write actions for accountability.

BAD: `POST /api/admin/users` accepts tier and `isAdmin` from request body without validation. An admin can set tier to any arbitrary string or `isAdmin` to a non-boolean. `GET /api/admin/users` uses `sortBy` query parameter directly in Prisma `orderBy` without allowlist validation, enabling ordering by sensitive columns like `stripeCustomerId`. `PATCH /api/admin/rooms` does not validate subject (unlike GET which does validate), allowing arbitrary subject values. Admin Export endpoint parses limit parameter but never applies it to the query, potentially returning all users and causing OOM.

REMEDY: Validate tier against `VALID_TIERS` in `POST`. Allowlist `sortBy` against safe columns. Validate subject in `PATCH` against `VALID_SUBJECTS`. Add `.take(limit)` to the export query. Cap limit at 100 in `GET` endpoints.

9. Scheduling, Homework, and Invoices

9.1 Schedule Panel (SchedulePanel.tsx - 533 lines)

GOOD: Tabbed layout separating upcoming and past lessons with counts on each tab. Create/Edit dialog with form validation. Status badges (Scheduled, Completed, Cancelled) with appropriate colors. Duration and student metadata shown per item. Loading skeleton consistent with MyRoomsPanel. Empty state with clear CTA. Cancel action available on upcoming items.

BAD: Cancel action has no confirmation dialog. One click on X icon immediately cancels a scheduled lesson. Form uses raw HTML label elements instead of `shadcn` Label component, missing `htmlFor` associations. `CalendarSync` component exists for Google Calendar and ICS downloads but is NOT integrated into the schedule list view. The edit dialog shares form state (creating boolean) between create and edit, which can display incorrect button text under parallel operations.

REMEDY: Add cancel confirmation dialog. Use `shadcn` Label in `renderForm`. Integrate `CalendarSync` component into schedule list items. Use separate loading states for create vs edit.

9.2 Homework Panel (HomeworkPanel.tsx - 746 lines)

GOOD: Status filtering across all homework states. Create and grade dialogs with proper form fields. Overdue detection highlights past-due homework. Student association links homework to the student model for proper agency tracking.

BAD: At 746 lines, this component should be split into smaller sub-components. Uses raw HTML labels in some form fields. Error swallowing in `fetchHomework` catch block. No character count on the description textarea.

REMEDY: Split into `HomeworkList`, `HomeworkCreateDialog`, `HomeworkGradeDialog` sub-components. Add error handling with `retry`. Add character count to textarea fields.

9.3 Invoice Panel (InvoicePanel.tsx - 619 lines)

GOOD: Create, send, and download invoice actions. Invoice status tracking (Draft, Sent, Paid, Overdue, Cancelled). Lesson hours and rate-per-hour billing calculation.

BAD: 619-line monolithic component. Hand-rolled HTML table instead of `shadcn` Table component. The invoice number generation API has a race condition: it does `findFirst` then increments without a transaction or unique constraint, so concurrent invoice creations for the same agency can produce duplicate numbers. Calendar ICS download for lessons has a broken agency owner access check that prevents agency owners from downloading ICS files for their sub-tutors lessons.

REMEDY: Split into smaller components. Use `shadcn` Table. Add unique constraint on `(agencyId, invoiceNumber)` or use `SELECT FOR UPDATE` in a transaction. Fix the ICS agency access check to verify `lesson.tutor.parentAgencyId === auth.userId` instead of checking the wrong field.

10. Parent Portal and Student Experience

10.1 Parent Portal (parent/[token]/page.tsx - 488 lines)

GOOD: Well-designed standalone portal with no sidebar or dashboard chrome. Agency branding (logo, name, colors) carried through from the agency profile. Four-tab layout (Schedule, Progress, Homework, Notes) provides comprehensive student visibility. Sticky header with student avatar and name. Proper empty states on all tabs: "No upcoming lessons scheduled" with Calendar icon, "All caught up!" with CheckCircle for homework, "No lesson notes yet" with StickyNote. Loading skeleton preserves full layout structure during data fetch.

BAD: Loading state lacks aria-live region for screen reader announcement. Error state lacks role="alert". Student email and name are hidden on mobile (hidden sm:block) with only an avatar icon shown, but the avatar div has no aria-label identifying who the portal belongs to. Homework count badge uses text-[9px] in a 16px circle, which is very small and hard to read. Agency logo uses img without onError fallback.

REMEDY: Add aria-live="polite" to loading container and role="alert" to error container. Add aria-label with student name to avatar div. Increase badge text size to at least text-[10px]. Add onError handler to agency logo to show fallback icon.

10.2 Student Dashboard (StudentDashboard.tsx - 614 lines)

GOOD: Mobile-first design with simplified experience. Quick join feature with URL/ID parsing. Getting started guide for new students. Skeleton loading state. Recent lessons with subject icons and duration display. Error boundary with retry capability.

BAD: Quick join has no validation feedback. If the user types an invalid room ID or a malformed URL, extractRoomId returns null but the UI shows nothing. The user clicks "Join" and nothing happens with no error message or visual feedback explaining why.

REMEDY: Show an inline error message when extractRoomId returns null and the user clicks "Join", explaining that the room ID or URL format is invalid.

11. Analytics and Data Visualization

11.1 Analytics Panel (AnalyticsPanel.tsx)

GOOD: Four stat cards with distinct gradient backgrounds for visual differentiation. Recharts integration with styled tooltips (rounded corners, shadows). Subject distribution shown with color-coded badges. Empty state with icon and clear CTA directing users to create rooms.

BAD: Chart colors are hardcoded (#059669, #e5e7eb, #6b7280) instead of using CSS variables, which breaks dark mode. No date range selector; always shows last 7 days and last 6 months with no user control. Error state is silently swallowed (`catch() => {}`) with no error UI shown to the user. Stats card borders use `border-0 shadow-sm` while other dashboard panels use `border border-gray-200`, creating visual inconsistency. The admin stats API endpoint has a `groupBy` bug: it groups by exact `createdAt` timestamp (millisecond precision) instead of calendar date, returning one group per unique timestamp rather than per-day aggregates.

REMEDY: Add an error state with retry button. Use CSS variable references for chart colors. Add date range filtering. Standardize card border styles across all panels. Fix admin stats `groupBy` to use `DATE(createdAt)` via raw SQL or Prisma raw query.

11.2 Agency Analytics (AgencyAnalyticsPanel.tsx)

GOOD: Tabbed layout with overview, per-tutor, and per-student views. Revenue tracking and lesson hour metrics. Subject distribution analytics.

BAD: 518-line monolithic component should be split. Toast import uses the wrong pattern (`const { toast } = useToast()` instead of `import { toast } from hooks`) creating inconsistency with other panels. Date parameters (`from`, `to`) are not validated before passing to `new Date()`, allowing Invalid Date objects.

REMEDY: Split into sub-components. Standardize toast imports. Add date validation (`z.string().datetime()` or `isNaN` check).

12. Cross-Cutting Patterns and Systemic Issues

12.1 Design System Consistency

Aspect	Status	Detail
Primary Color	Consistent	#059669 (emerald-600) as --primary throughout
Gradient CTA	Consistent	gradient-primary (emerald to teal) on all primary buttons
Typography	Consistent	Geist Sans, consistent scale and weights
Spacing	Consistent	Standardized gap-2/3/4/6, card padding p-4/5/6
Border Radius	Consistent	rounded-xl buttons, rounded-2xl cards, rounded-full badges
Dark Mode	BROKEN	80%+ components hardcode light colors, no dark: variants
Loading Patterns	Inconsistent	3 different patterns: Skeleton, animate-pulse, custom spinner
Toast Imports	Inconsistent	Mix of useToast() hook and named import
Shadow Usage	Consistent	shadow-sm cards, shadow-lg CTAs, shadow-{color} stat cards
Icons	Consistent	100% Lucide React throughout

12.2 Accessibility Summary

Category	Status	Notes
Skip Navigation	Present	Root layout has sr-only skip-to-content link with focus:not-sr-only
Semantic HTML	Good	Proper header, main, footer, nav, table usage
ARIA Attributes	Incomplete	Missing role="alert" on error states, missing aria-label on loading spinners
Focus Management	Good	Skip link works, outline-ring focus indicators in base styles
Color Contrast	Good	Emerald on white passes WCAG AA
Reduced Motion	Good	@media (prefers-reduced-motion) disables all animations globally
Image Fallbacks	Incomplete	Agency logos missing onError handlers in parent portal and invite page
Form Labels	Partial	Some forms use raw label without htmlFor; others use shadow Label properly

12.3 N+1 API Call Pattern

A systemic N+1 API call pattern affects RecordingsPanel and LessonNotesPanel. Both components first fetch all rooms via GET /api/room, then iterate through ended rooms and fire individual API calls per room to fetch recordings or lesson notes. With a moderate number of rooms (50+), this creates 51+ API requests on every panel mount, significantly degrading load time and increasing server load. The recommended fix is to create batch endpoints that accept a list of room IDs and return all related data in a single response. Alternatively, the existing room

list endpoint could include embedded recording/note data, or the panels could use a single aggregated endpoint.

12.4 Silent Error Swallowing

A pervasive pattern across the codebase: catch blocks with empty bodies or commented-out error handling. The root `page.tsx` has 5+ silent catch blocks that swallow auth failures, profile fetch errors, and admin check failures. `TemplatesPanel`, `HomeworkPanel`, and `ResourceLibraryPanel` all have empty catch blocks in critical operations. `AnalyticsPanel` catches errors with no user-facing error state. This makes debugging extremely difficult in production and leaves users confused when operations silently fail. Every catch block should at minimum log to `console.warn` or `console.error`, and user-facing operations should show an error toast or inline error message.

13. Priority Remediation Roadmap

Phase 1: Critical Fixes (Immediate - 1-2 days)

#	Finding	Severity	Effort	Action
1	CSRF conditional bypass	CRITICAL	30 min	Reject when cookie/header missing on mutations
2	Recording secret instability	CRITICAL	15 min	Fail at startup if env var missing
3	LiveKit webhook fallthrough	CRITICAL	5 min	Move return outside egress_failed block
4	Rate limit IP bypass	HIGH	1 hr	Apply default limit for unknown IPs

Phase 2: High Priority (1 week)

#	Finding	Severity	Effort	Action
5	No destructive confirmations	HIGH	2-3 hr	Add AlertDialog to End Room, Cancel Lesson, Delete actions
6	Admin input validation gaps	HIGH	3 hr	Add allowlists for tier, sortBy, subject across all admin endpoints
7	Broken agency ICS access	HIGH	30 min	Fix parentAgencyId check to use tutor.parentAgencyId
8	Invoice number race condition	HIGH	1 hr	Add unique constraint or SELECT FOR UPDATE
9	CSRF timing-safe comparison	HIGH	15 min	Use crypto.timingSafeEqual (pattern exists in codebase)
10	Admin export limit not applied	HIGH	5 min	Add .take(limit) to query

Phase 3: Medium Priority (2-3 weeks)

#	Finding	Severity	Effort	Action
11	Dark mode broken	MEDIUM	2-3 days	Migrate all bg-white/text-gray-* to semantic CSS tokens
12	N+1 API in Recordings/Notes	MEDIUM	2-4 hr	Create batch endpoints
13	Loading pattern inconsistency	MEDIUM	2-3 hr	Standardize on Skeleton component across all panels
14	Missing ARIA attributes	MEDIUM	2 hr	Add role="alert" and aria-label across all panels
15	Missing rate limits on AI/fingerprint	MEDIUM	1 hr	Apply existing checkRateLimit to unprotected endpoints
16	Extract auth from LandingPage	MEDIUM	1-2 hr	Create AuthDialog component
17	TemplatesPanel disabled play button	MEDIUM	1-2 hr	Implement template loading or remove dead UI
18	Student import N+1 queries	MEDIUM	2 hr	Batch process with createMany/updateMany

Phase 4: Polish and Low Priority (Ongoing)

#	Finding	Severity	Effort	Action
19	Password visibility toggle	LOW	30 min	Add eye icon toggle to password fields
20	Contact page placeholders	LOW	15 min	Update business name, address, GSTIN
21	Toast import inconsistency	LOW	30 min	Standardize across all files
22	Page thumbnails in sidebar	LOW	2-3 days	Add canvas page preview thumbnails
23	Service worker dead code	LOW	15 min	Register sw.js or remove the file
24	Component file size reduction	LOW	2 days	Split 500+ line components into sub-components

14. Conclusion

Superboard is a well-architected tutoring platform with a comprehensive feature set that covers the full lifecycle of online tutoring: from marketing and onboarding, through real-time collaborative lessons with video, whiteboard, and AI tools, to post-lesson management with homework, notes, invoicing, and analytics. The agency management tier adds significant depth with sub-tutor coordination, student rosters, branding customization, and resource sharing.

The engineering quality is above average for a startup product. The security architecture demonstrates deliberate hardening with multiple layers including CSP nonces, double-submit CSRF, HMAC-signed recording URLs, SSRF protection, and tiered rate limiting. The UI design system is cohesive with consistent emerald branding, proper typography hierarchy, and responsive layouts that work across mobile and desktop.

The most impactful improvements come in three areas. First, fixing the 4 Critical security findings (CSRF bypass, recording secret instability, LiveKit webhook fallthrough, and rate limit bypass) which pose immediate risk. Second, adding destructive action confirmations across the application, as the current single-click destructive actions create a poor user experience and risk accidental data loss. Third, either fully implementing dark mode with semantic CSS tokens or removing the toggle, since the current half-baked implementation damages credibility.

Addressing the 4 Critical, 15 High, and key Medium findings outlined in this report would bring Superboard to a production-ready quality level suitable for scaling the user base and attracting enterprise agency clients.